COMISIÓN A+F+C: Ejercicio Práctico en Papel

El siguiente ejercicio debe ser resuelto en hoja con lapicera/lapiz. Todas las hojas deben tener nombre y apellido. Tener en cuenta la legibilidad de la solución entregada, ya que de no comprenderse lo escrito no se podrá corregir. Deben entregarse todas las hojas que contengan código asociado a la solución. Recuerde que si hace uso de funciones auxiliares, debe incluirlas y/o codificarlas según corresponda.

Tiempo de Resolución: 90 minutos.

Puntaje Requerido: 20/40 puntos.

Consigna:

Un texto pueden ser visto como un conjunto de oraciones ordenadas. La forma en la que cada oración puede ser representada es por medio de su ordinal (posición dentro del conjunto comenzando con el valor 1) y del string que mantiene su contenido. Se sabe que, como máximo, en un texto existen 100 oraciones.
A Federico se le pidió que analice el contenido de un apunte y, para esto, elaboró una pila que contiene las oraciones de dicho texto siguiendo el siguiente criterio: cada elemento que se agrega a la pila corresponde a la oración más larga que aún existe en el texto bajo análisis.
En este contexto, se le solicita:

a) Defina todas las estructuras de datos necesarias para representar un texto.

b) Defina todas las estructuras de datos necesarias para representar la pila elaborada por Federico en relación a un texto que sigue la definición del ítem a).

c) Defina e implemente la función *generarTextoOriginal* la cual recibe la pila armada por Federico y devuelve el texto original (es decir, el conjunto de oraciones ordenadas según su ordinal).

d) Defina e implemente la función *oracionesCortas* la cual recibe la pila armada por Federico y un valor entero N. La función devuelve la cantidad de oraciones que tienen una longitud menor a N.

e) Defina e implemente una función de complejidad log N que, recibiendo como argumento un texto, imprima por pantalla el contenido de las oraciones ubicadas en los ordinales definidos por 2^k partiendo de k=0 y considerando 2^k menor a la cantidad de oraciones en el texto. Justifique porque su algoritmo tiene la complejidad solicitada.

Nota: Para la resolución del problema el alumno puede codificar todas las funciones que considere necesarias. La cantidad de parámetros formales asociados a cada función debe ser definida por el alumno. El puntaje final obtenido tendrá en cuenta la eficiencia de la estrategia de resolución elegida.

AEDD- EXAMEN FINAL - 6/7/2022

Ejercicio de Práctica en Papel - Regulares

El siguiente ejercicio debe ser resuelto en hoja con lapicera/lapiz. Todas las hojas deben tener nombre y apellido. Tener en cuenta la legibilidad de la solución entregada, ya que de no comprenderse lo escrito no se podrá corregir. Deben entregarse todas las hojas que contengan código asociado a la solución. Recuerde que si hace uso de funciones auxiliares, debe incluirlas y/o codificarlas según corresponda.

Tiempo de Resolución: 90 minutos.

Puntaje Requerido: 20/40 puntos.

Consigna:

Con un alto nivel de abstracción, podemos pensar que una SERIE queda identificada por su nombre (tipo cadena de caracteres) y un conjunto de TEMPORADAS. A su vez, una TEMPORADA puede ser vista como un conjunto de CAPITULOS. No existe un limite para la cantidad de TEMPORADAS, pero si para la cantidad de CAPITULOS en una TEMPORADA (máximo 30). Cada CAPITULO tiene asociado un título (tipo cadena de caracteres), una duración en minutos (dato tipo entero) y una descripción (tipo string).

En base a estas definiciones, se le solicita:

a) Defina todas las estructuras de datos necesarias para representar el TDA SERIE.

Luego, como parte del TDA SERIE, se le solicita:

b) Defina e implemente la función *capituloMasLargo* la cual recibe una SERIE y devuelve la duración en horas y minutos del CAPITULO que tiene la mayor duración.

c) Defina e implemente la función *temporadaMasCorta* que recibe una SERIE e imprime por pantalla los titulos de los CAPITULOS que forman parte de la TEMPORADA con menor cantidad de CAPITULOS.

d) Defina e implemente la función recursiva *toUpperName* que recibe una SERIE y modifica su nombre de forma tal que todos los caracteres que lo componen se encuentren en maýusculas.

e) Defina e implemente la función *imprimirInformación* la cual recibe una SERIE y visualiza en pantalla su información general (nombre y cantidad de TEMPORADAS) junto con la cantidad de CAPITULOS asociados a cada TEMPORADA.

Importante: Para la resolución del problema el alumno puede codificar todas las funciones que considere necesarias. La cantidad de parámetros formales asociados a cada función debe ser definida por el alumno. El puntaje final obtenido tendrá en cuenta la eficiencia de la estrategia de resolución elegida.

AEDD- EXAMEN FINAL - 23/5/2022

Ejercicio de Práctica en Papel - Regulares

Un TICKET puede ser visto como un listado de ITEMS a ser facturados en una CAJA (valor entero). Cada ITEM contiene un PRODUCTO y una CANTIDAD DE UNIDADES A FACTURAR. A su vez, un PRODUCTO queda definido por su CODIGO (valor entero), su DESCRIPCIÓN (tipo string) y su PRECIO UNITARIO. Se sabe que no existe un límite para la cantidad de ITEMS a facturar en un TICKET, que los CODIGOS de PRODUCTO corresponden a números comprendidos entre 0-49 (inclusive) y que una CAJA queda identificada por un valor entero comprendido en el rango 0-9 (inclusive).

En base a estas definiciones, se le solicita:

a) Defina todas las estructuras de datos necesarias para representar un conjunto de 200 TICKETS facturados.

b) Defina e implemente la función *mayorTicket* la cual recibe los 200 TICKETS y devuelve el TICKET que tiene la mayor facturación.

c) Defina e implemente la función *cantidadDeUnidades* que recibe un TICKET y devuelve el TOTAL DE UNIDADES facturadas.

d) Defina e implemente la función recursiva *ticketsMenorA* que recibe los 200 TICKETS y un valor punto flotante MENOR. La función debe devolver la cantidad de TICKETS cuya facturación ha sido inferior a MENOR.

e) Defina e implemente la función *cargarVentas* la cual recibe un conjunto de TICKETS facturados y se encarga de cargar en una matriz de enteros la cantidad de unidades facturadas de cada producto en cada una de las cajas. Cada fila de la matriz corresponde a un producto, mientras que cada columna corresponde a una caja. Como resultado de ejecutar la función, se debe visualizar en pantalla en contenido de la matriz.

Importante: Para la resolución del problema el alumno puede codificar todas las funciones que considere necesarias. El puntaje final obtenido tendrá en cuenta la eficiencia de la estrategia de resolución elegida.

Algoritmos y Estructuras de Datos
Examen Final ***25/10/2023***

Ejercicio de Práctica en Papel

Nombre y Apellido: .....

Año de Cursado:.....

El siguiente ejercicio debe ser resuelto en esta hoja con lapicera/lápiz. Puede utilizar hojas auxiliares. Todas las hojas deben tener nombre y apellido. Tener en cuenta la legibilidad de la solución entregada, ya que de no comprenderse lo escrito no se podrá corregir. Deben entregarse todas las hojas que contengan código asociado a la solución. Recuerde que si hace uso de funciones auxiliares, debe incluirlas y/o codificarlas según corresponda.

Tiempo de Resolución: 90 minutos.
Puntaje Requerido: 24/40 puntos.

Consigna:g

Una COLA DE PRIORIDADES es un Tipo de Dato Abstracto similar a una cola en la que los elementos tienen adicionalmente, una prioridad asignada. En una COLA DE PRIORIDADES un elemento con mayor prioridad será desencolado antes que un elemento de menor prioridad. Si dos elementos tienen la misma prioridad, se desencolarán siguiendo el orden de cola.

En nuestro enunciado, sabemos que únicamente existen 3 tipos de prioridad para un ELEMENTO: 1 (más prioritaria), 2 (prioridad intermedia), 3 (menos prioritario). Luego, se propone elaborar una definición de COLA DE PRIORIDADES bajo el siguiente esquema:

```
struct ColaDePrioridades {
      Cola colaPrioridad1, colaPrioridad2, colaPrioridad3;
};
```

donde cada subcola mantiene los elementos encolados para cada prioridad, con un máximo de 100 en cada caso. Luego, por ejemplo, si la `colaPrioridad1` se encuentra vacia, el primer elemento a remover de la estructura será el primer elemento de la `colaPrioridad2`. A su vez, una inserción en una `ColaDePrioridades` se efectúa en la cola asociada a su prioridad, según el orden de cola.

En base a estas definiciones, se le solicita:

a) Defina la interfaz del TDA `Cola`, el cual actúe como estructura de datos de ELEMENTOS definidos de la siguiente manera:

```
struct Elemento {
      int valor;
      char letra;
};
```

b) Defina el TDA `ColaDePrioridades` haciendo uso del TDA `Cola`. Para este TDA, se solicita incluya la implementación de las funciones: `crearColaDePrioridades()`, `encolarElemento()`, `desencolarElemento()`. Tener en cuenta que:

- La prioridad de un `Elemento` queda definida según la posición de `letra` en el alfabeto sumado a `valor` y, sobre ese resultado, calculando el resto de la división por 3.

- De ser necesario, puede completarse la definición e implementación del TDA Cola.

c) Haciendo uso de los TDA definidos en los ítems a) y b), implemente la función recursiva:

```
Cola getColaInvertida(ColaDePrioridades c);
```

Esta función devuelve los elementos que pertenecen a una `ColaDePrioridades` encolados en orden invertido en una `Cola`.

d) Sobre la `Cola` obtenida en el ítem c), defina e implemente una función que genere una lista ordenada ascendentemente de los valores que acompañan los elementos que contienen la letra M.

Importante: Para la resolución del problema el alumno puede codificar todas las estructuras de datos y funciones que considere necesarias. Los campos de las estructuras deben respetar lo enunciado en la consigna. En los casos en los que no se indica un prototipo, los parámetros formales de las funciones (cantidad y tipo) deben definirse según los objetivos propuestos. El puntaje final obtenido tendrá en cuenta la eficiencia de la estrategia de resolución elegida.